

Lecture 11

- Functional programming (FP)
 - pure functions
 - with no side-effects
 - function as the 1st class citizen
 - lambdas, i.e., anonymous functions
 - often Arrow functions (but need not to be)
 - where **this** refers to global scope (window)
 - no arguments object
- Higher-order functions
 - map()
 - filter()
 - reduce()

How is JavaScript as lang? (Kickoff)

*The syntax seems somewhat easy. To me, at this stage, it appears to be somewhat of a **functional programming** language, but I'm not sure to what extent. Basically at this point I think it will be an easy language to use, even though I have almost no previous experience with it.*

*One person from my team said that Javascript feels like Python but forced. I also did the first exercise in Javascript and found it a bit difficult to understand because, for instance, **functions can be variables**. We also thought that is a bit weird that numbers and strings can be equal. Also, the concept of hoisting is a bit weird.*

*JavaScript is a bit confusing at the moment. **Functions can be coded in multiple ways**. No strict type operations. Variable scope messy.*

*We think it's overall a powerful language but it would be easy to have hidden bugs such as indexing errors on arrays. It seems harder to **get error messages on JavaScript than most other languages because it's so flexible**.*

*I've had a bit experience in writing JS and I see it as **hard to debug without Type Script and the node modules**. However, it has many different features and truly, my imagination is the limit.*

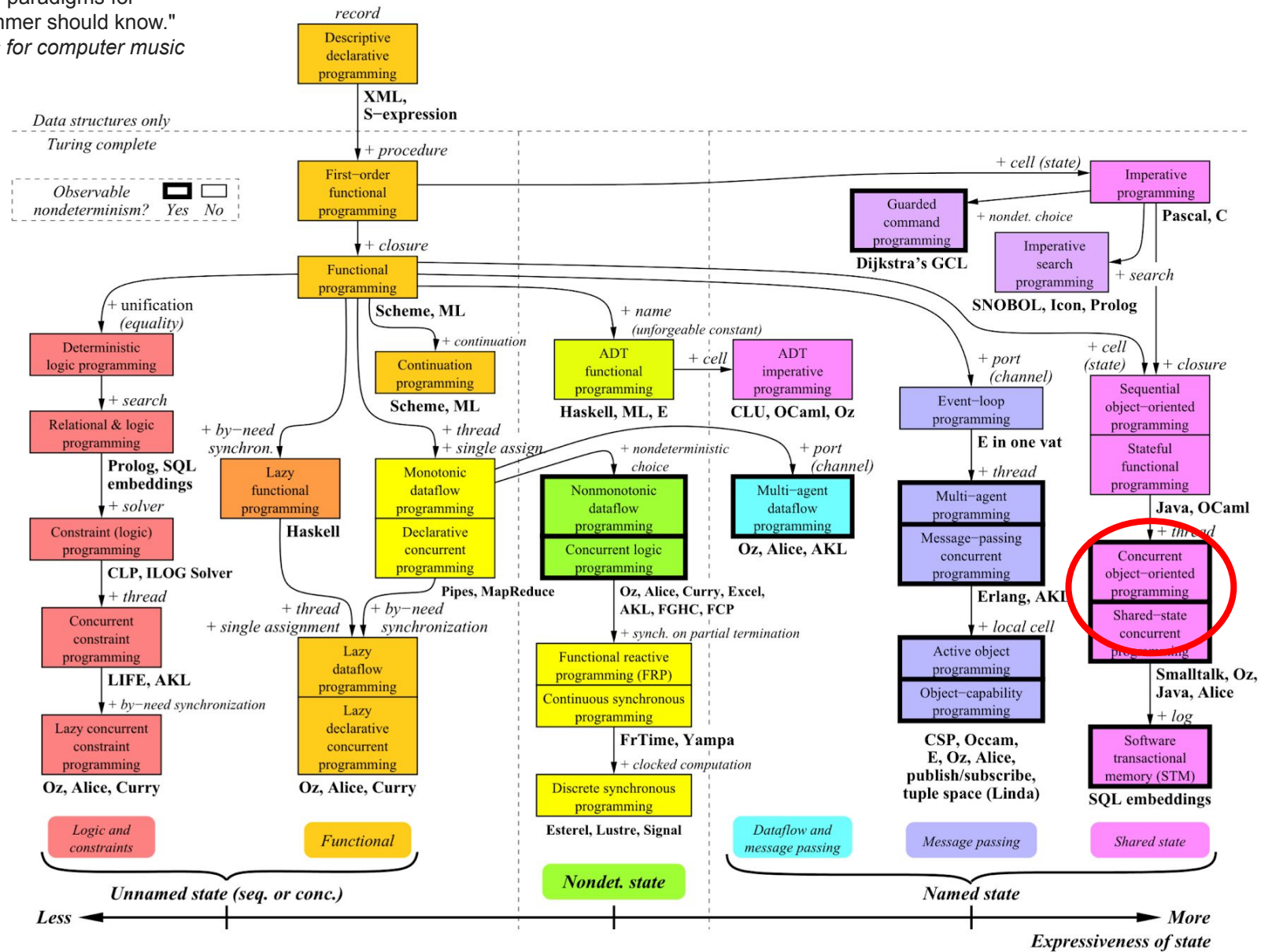


Figure 2. Taxonomy of programming paradigms

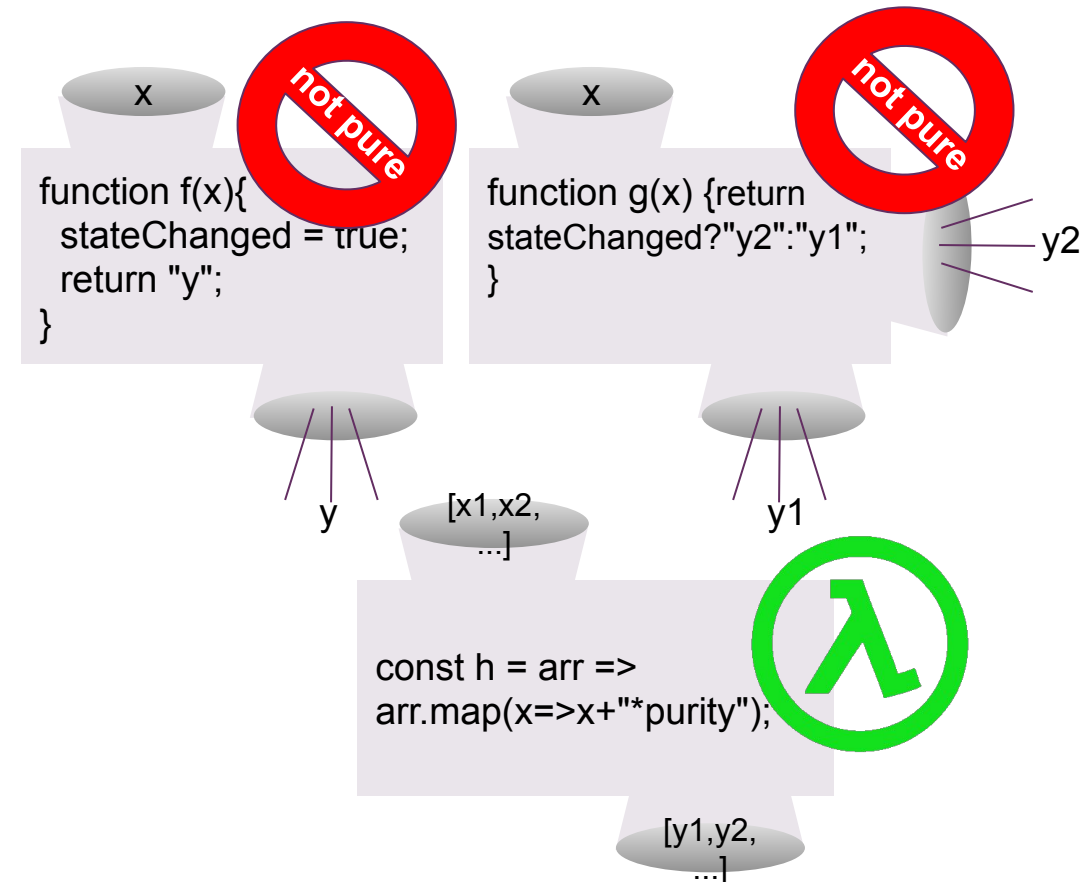


No side effects with pure functions

A pure function

- acts like a function in math
- returns always the same output for an input, i.e., is idempotent
 - thus the return value must not rely on indeterminate values, such as `Math.random()` or `Date`
- causes no side-effects, i.e., changes to state:
 - changes to its parameters passed by reference
 - modifications out of its scope, e.g., to global variables
 - by AJAX, timers, DOM manipulations
 - logging to a console
- had the impure actions to be executed for desired functionality, purity principles would still be good to be kept in mind as much as possible

```
let stateChanged = false; //global variable
const y1= g(x);           //"y1" relies on global variable
const y = f(x);           //"y" changes the state
const y2 = g(x);          //"y2" returns then a different value
```



Provides a secure, testable, and cacheable building block!

Function as the first class citizen

means that function can be, e.g.,

- passed as an argument
- set as a variable
- returned by a function
- set as a callback (cb)

Met-befores during WebCAIS:

- element.addEventListener(type, **cb**)
- setTimeout(**cb**, timeout)
- custom higher-order functions

```
const a = par => `${par} is the best`;  
const b = func => func("Pure");  
const c = (func, par) => func(par);  
b(a);  
c(a, "Pure");
```

Arrow function

- new in ES6, intended to
 - shorten the syntax
 - remove ambiguity about the current *this* object

Conventional function as an expression (no name)

```
const sayHello = function (greeting, sname) {  
    console.log(greeting + " " + sname);  
}
```

Arrow function

```
const sayHello = (greeting, sname) => console.log(`${greeting} ${sname}`);
```

Arrow function syntax

```
(param1, param2) => { statement1; statement2 }
```

- if one parameter only, no round brackets

```
param1 => {statement1; statement2}
```

- if only one statement, no curly brackets

```
(param1, param2) => statement1;
```

- if no parameters, round brackets **are** needed

```
() => statement1;
```

- **this** refers to the global scope
- What does the following function do? `item=>item`

this in arrow functions

Conventional function

```
const sname = "Boris";

function sayHelloforAll(greet){
  console.log(greet+ " " + this.sname);
}
sayHelloforAll("Hello");    // Hello Boris

const s2 = {
  sname:"Martta",
  sayHello: sayHelloforAll
}
const s3 = {
  sname:"Håkan",
  sayHello: sayHelloforAll
}
s2.sayHello("Hei");        // Hei Martta
s3.sayHello("Hejsan");     // Hejsan Håkan
```

Arrow function

```
const sname = "Boris";

let sayHelloforAll = greet =>
  console.log(greet + " " + this.sname);

const s2 = {
  sname:"Martta",
  sayHello: sayHelloforAll
}

const s3 = {
  sname:"Håkan",
  sayHello: sayHelloforAll
}

s2.sayHello("Hei");        // Hei Boris
s3.sayHello("Hejsan");     // Hejsan Boris
```

No arrow function constructor

```
let Student = (sname, age, homeTown) => {  
  this.sname = sname;  
  this.age = age;  
  this.homeTown = homeTown;};  
  
const s2 = Student("Anu", 27, "Orivesi");
```

- s2 is undefined
- However, sname, age and homeTown are defined as global variables

Arguments of an enclosing scope

Conventional function

```
// implicit arguments binding
function test(n) {
  const f = () => arguments[0] + n;
  // arguments[0] is n
  return f();
}
```

```
test(3); // 6
```

Arrow function

```
const arguments = [1, 2, 3];
const arr = () => arguments[0];
// ref. to a global variable
```

```
arr(); // 1
```

No arguments in arrow function

- In conventional functions
 - the number of parameters may vary, but the variation can be handled with `arguments`
- In arrow functions
 - NO own `arguments` object
 - `arguments` may refer to `arguments` of the enclosing scope depending on the situation

Array functions for FP

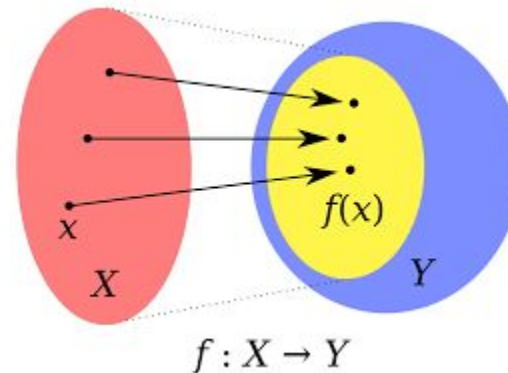
- `Array.map()`
- `Array.filter()`
- `Array.reduce()`

- used to replace loops (for, while)
- often implemented by calling arrow functions
- are so called 'higher-order functions': functions calling functions

Array.map() shortly

```
let new_array = arr.map(function callback( currentValue[, index[,  
array]]) {  
    // return element for new array  
},[, thisArg]); thisArg - optional, value for 'this' in callback
```

- Callback function is called for each item in array
- A new array is constructed as a return value
 - The array should be of the same length as the array to be mapped
 - To return an array with different length is map Anti-pattern
- The math analog of map() is a function
 - that maps a domain to a range



students array to be mapped

```
const students = [  
  {  
    "name": "Teemu Teekkari",  
    "studentId": "tuni12345",  
    "email": "teemu.teekkari@tuni.fi"  
  },  
  {  
    "name": "Teppo Testaaja",  
    "studentId": "tuni23456",  
    "email": "teppo.testaaja@tuni.fi"  
  },  
  {  
    "name": "Jaakko Javascript",  
    "studentId": "tuni34567",  
    "email": "jaakko.javascript@tuni.fi"  
  }  
]
```

Array.map() has certain benefits over for loops

for loop

```
let names;  
const l = students.length;  
for (let i = 0; i < l; i++) {  
  names.push(students[i].name);  
}  
console.log(names);
```

Array.map()

```
const names =  
students.map(std => std.name);  
// name implicitly returned
```


Array.filter() shortly

```
let newArray = arr.filter(callback(element[, index, [array]])[,  
thisArg]) //thisArg - optional, value for 'this' in callback
```

- Callback function is called for each array item
- callback returns
 - **true** to keep the element
 - **false** to discard the element
- Array.filter() returns a new filtered array of kept elements

students array to be filtered

```
const students = [  
  {  
    "name": "Teemu Teekkari",  
    "studentId": "tuni12345",  
    "email": "teemu.teekkari@tuni.fi",  
    "major": "web&cloud"  
  },  
  {  
    "name": "Teppo Testaaja",  
    "studentId": "tuni23456",  
    "email": "teppo.testaaja@tuni.fi",  
    "major": "web&cloud"  
  },  
  {  
    "name": "Jaakko Javascript",  
    "studentId": "tuni34567",  
    "email": "jaakko.javascript@tuni.fi",  
    "major": "mathematics"  
  }  
]
```

Array.filter() has certain benefits over for loops

for loop

```
let webMajors=[];  
const MAJOR = "web&cloud";  
const l = students.length;  
for (let i = 0; i < l; i++) {  
  if (students[i].major===MAJOR)  
    webMajors.push(students[i].name);  
}  
console.log(webMajors);
```

Array.filter()

```
const MAJOR = "web&cloud";  
const webMajors =  
  students.filter(std =>  
    std.major === MAJOR);
```

Array.reduce() shortly

```
arr.reduce(callback( accumulator, currentValue[, index[, array]]  
)[, initialValue])
```

- Callback function is called for every element in the array
 - Accumulator accumulates return values
- often used to fold an array of values into a single value, e.g., sum or avg
- Initial value is mandatory for an array of objects
 - 99% of the time you handle an array of objects

shopping cart to be reduced

```
const shoppingCart = [  
  {  
    "name": "toothbrush",  
    "category": "hygiene",  
    "price": 2.99  
  },  
  {  
    "name": "offBrandCoke",  
    "category": "foodAndDrinks",  
    "price": 0.99  
  },  
  {  
    "name": "offBrandHamburger",  
    "category": "foodAndDrinks",  
    "price": 1.49  
  },  
  {  
    "name": "offBrandFries",  
    "category": "foodAndDrinks",  
    "price": 1.49  
  }  
]
```

Array.reduce() has certain benefits over for loops

for loop

```
let total = 0;

for (let i = 0; i <
shoppingCart.length; i++)
{
    total = total +
shoppingCart[i].price;
}
```

Array.reduce()

```
const total =
shoppingCart.reduce(( sum, item ) =>
    sum + item.price, 0)
```

shopping cart reduced by category 1/2

```
const categories = ["hygiene", "foodAndDrinks"];

const pricesByCategory = categories.map(category =>
  shoppingCart.filter(item => item.category === category)
    .reduce((priceObject, item) => {
      priceObject[item.category] =
        (priceObject[item.category] || 0) + item.price;
      return priceObject;
    }, {}));

console.log(pricesByCategory)
// [ { hygiene: 2.99 }, { foodAndDrinks: 3.96 } ]
```

shopping cart reduced by category 2/2

```
const pricesByCategory2 = shoppingCart.reduce((obj,item) => {  
  obj[item["category"]]=(obj[item["category"]] || 0)+item["price"];  
  return obj;}, {})
```


Promoted FP features for robustness

- **const**
 - to pave the way for immutability
 - more robust coding
 - easier bug tracking
- lambdas
- higher-order functions, e.g., **map()**, **filter()** and **reduce()**
 - make code more expressive
 - why Array functions, e.g., `map()`, are better than for loop:
 - return a new array instead of modifying the old one
=> immutability
 - no reassignable counters, thereby no accidental global vars
 - no such errors as infinite loops or off-by-one either